

宁波大学
NINGBO UNIVERSITY

实 验 报 告

实验 4 SQL 的视图

姓名：	顾凯琪
学号：	226001624
课程名称：	数据库原理与应用
所在学院：	信息科学与工程学院
专业班级：	22 计算机
任课教师：	辛宇

实验项目名称	实验 4 SQL 的试图

一、 实验目的

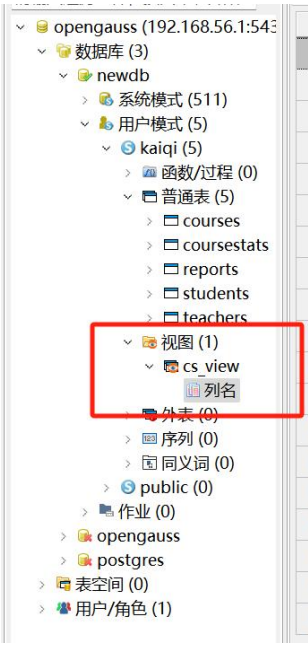
熟悉 SQL 支持的有关视图的操作，能够熟练使用 SQL 语句来创建需要的视图，对视图进行查询和删除视图。定义数据库外模式，并能使用所创建的视图实现数据管理。理解和掌握视图消解执行原理，掌握可更新视图和不可更新视图的区别。理解物化视图。

二、 实验内容及过程、结果

1) 创建一个行列子集视图 CS_View，给出选课成绩合格的学生的编号、教师编号、所选课程号和该课程成绩，并查询该视图。

通过 CREATE VIEW 创建视图 CS_View，筛选出成绩合格（成绩大于等于 60）的学生的编号、教师编号、课程编号及成绩。然后通过 SELECT 语句查询该视图。

```
1 CREATE VIEW CS_View AS
2 SELECT Sno, Tno, Cno, Score
3 FROM Reports
4 WHERE  Score>= 60;
5
6 SELECT * FROM CS_View;
7
```

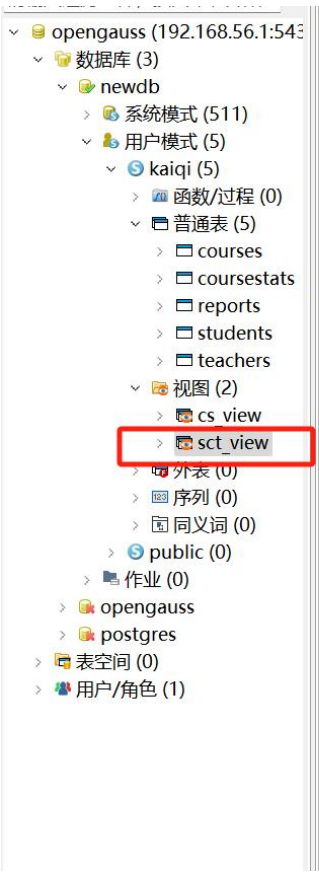


	sno	tno	cno	score
1	S01	T01	C01	83.0
2	S02	T01	C01	75.0
3	S02	T05	C05	70.0
4	S02	T04	C06	83.0
5	S02	T05	C07	90.0
6	S02	T01	C08	83.0
7	S02	T02	C09	77.0
8	S02	T07	C10	83.0
9	S02	T06	C11	88.0
10	S03	T01	C08	63.0
11	S03	T02	C02	93.0
12	S03	T01	C01	78.0
13	S04	T06	C06	89.0
14	S04	T05	C05	93.0
15	S26	T04	C04	86.0
16	S52	T07	C10	91.0
17	S52	T06	C11	90.0
18	S52	T01	C08	64.0
19	S52	T02	C09	81.0

2) 创建基于多个基本表的视图 **SCT_View**，这个视图由学生姓名和他所选修的课程名及讲授该课程的教师姓名构成，并查询该视图。从视图 **SCT_View** 中查询所有选修课程“数据库原理及应用”的学生姓名。

该视图从 Students、Courses 和 Teachers 三个基本表中连接查询学生姓名、课程名和授课教师姓名。这种多表连接视图将不同表中的信息组合成一张虚拟表，便于进一步查询。

```
3  FROM Students S
4  JOIN Reports R ON S.Sno = R.Sno
5  JOIN Courses C ON R.Cno = C.Cno
6  JOIN Teachers T ON R.Tno = T.Tno;
7
8  -- 查询视图
9  SELECT * FROM SCT_View;
10
11 -- 查询选修课程"数据库原理及应用"的学生姓名
12 SELECT Sname FROM SCT_View
13 WHERE Cname = '数据库原理及应用';
14
```



	sname	cname	tname
1	王建平	C++	刘涛
2	刘华	C++	刘涛
3	刘华	英语	刘涛
4	刘华	UML	吴碧艳
5	刘华	数字生活	吴碧艳
6	刘华	算法分析与设计	张宁雅
7	刘华	数据结构与算法	张宁雅
8	刘华	数据库原理及应用	叶帅
9	刘华	计算机组成原理	叶帅
10	刘华	体育1	杨光美
11	刘华	音乐鉴赏	程潜
12	范林军	C++	刘涛
13	范林军	英语	刘涛
14	范林军	UML	吴碧艳
15	李伟	数据库原理及应用	叶帅
16	李伟	数据结构与算法	杨光美
17	黄河	算法分析与设计	张宁雅
18	黄河	音乐鉴赏	程潜
19	长江	英语	刘涛
20	长江	数字生活	吴碧艳
21	长江	数据库原理及应用	叶帅
22	长江	体育1	杨光美
23	长江	音乐鉴赏	程潜

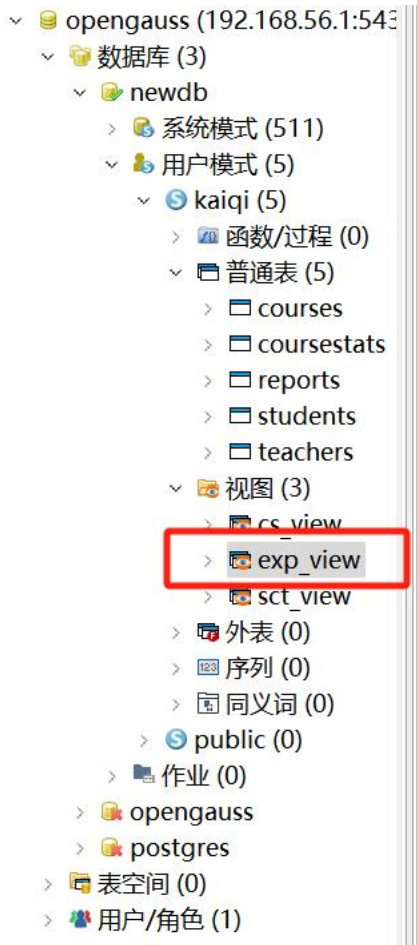
3) 创建带表达式的视图 **EXP_View**，由学生姓名及所选课程名和所有课程成绩都比原来多 **5** 分这几个属性组成，并查询该视图。

该视图使用表达式对原始数据进行变换，创建一个新的虚拟列，将每个学生的课程成绩增加 **5** 分。这种视图用于直接在结果集上应用计算逻辑。

```
CREATE VIEW EXP_View AS
SELECT S.Sname, C.Cname, R.Score + 5 AS Adjusted_Grade
FROM Students S
JOIN Reports R ON S.Sno = R.Sno
JOIN Courses C ON R.Cno = C.Cno;

-- 查询视图
SELECT * FROM EXP_View;

```

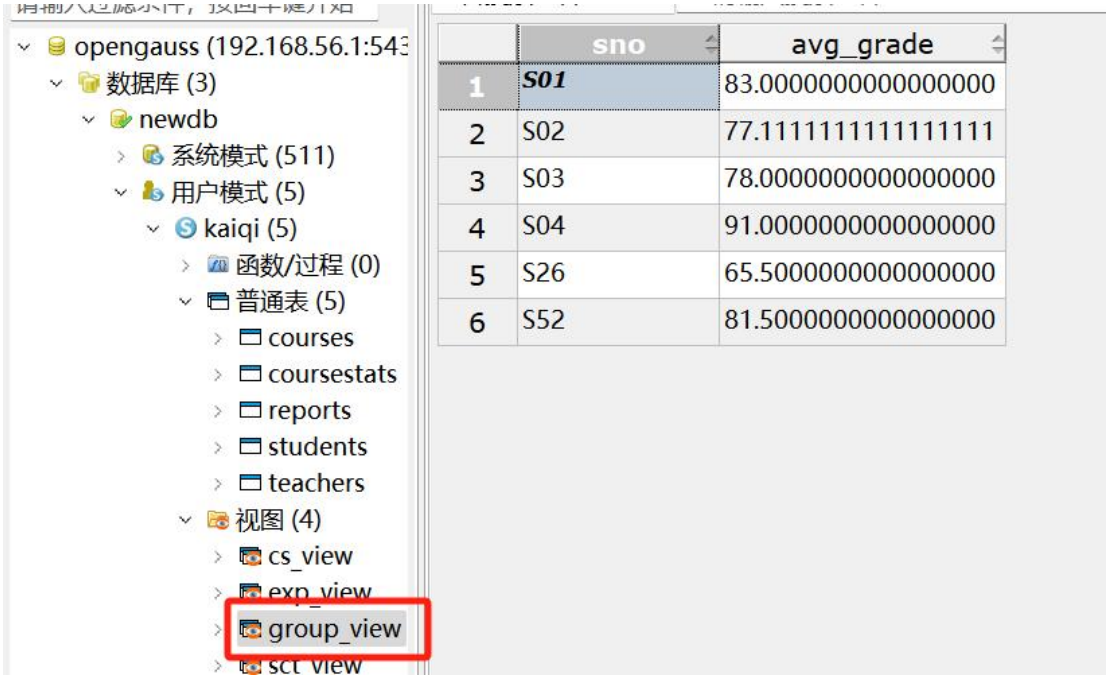


	sname	cname	adjusted_gr...
1	王建平	C++	88.0
2	刘华	C++	80.0
3	刘华	英语	88.0
4	刘华	UML	50.0
5	刘华	数字生活	82.0
6	刘华	算法分析与设计	[NULL]
7	刘华	数据结构与算法	88.0
8	刘华	数据库原理及应用	75.0
9	刘华	计算机组成原理	95.0
10	刘华	体育1	93.0
11	刘华	音乐鉴赏	88.0
12	范林军	C++	83.0
13	范林军	英语	68.0
14	范林军	UML	98.0
15	李伟	数据库原理及应用	98.0
16	李伟	数据结构与算法	94.0
17	黄河	算法分析与设计	91.0
18	黄河	音乐鉴赏	50.0
19	长江	英语	69.0
20	长江	数字生活	86.0
21	长江	数据库原理及应用	[NULL]
22	长江	体育1	95.0
23	长江	音乐鉴赏	96.0

4) 创建分组视图 **Group_View**，将学生的学号及他的平均成绩定义为一个视图，并查询该视图。

此操作通过分组（GROUP BY）和聚合函数（AVG()）创建一个视图，显示学生的学号及其平均成绩。分组视图非常适合用于汇总信息，如统计平均值、总数等。

```
1 CREATE VIEW Group_View AS
2 SELECT Sno, AVG(Score) AS Avg_Grade
3 FROM Reports
4 GROUP BY Sno;
5
6 -- 查询视图
7 SELECT * FROM Group_View;
8
```

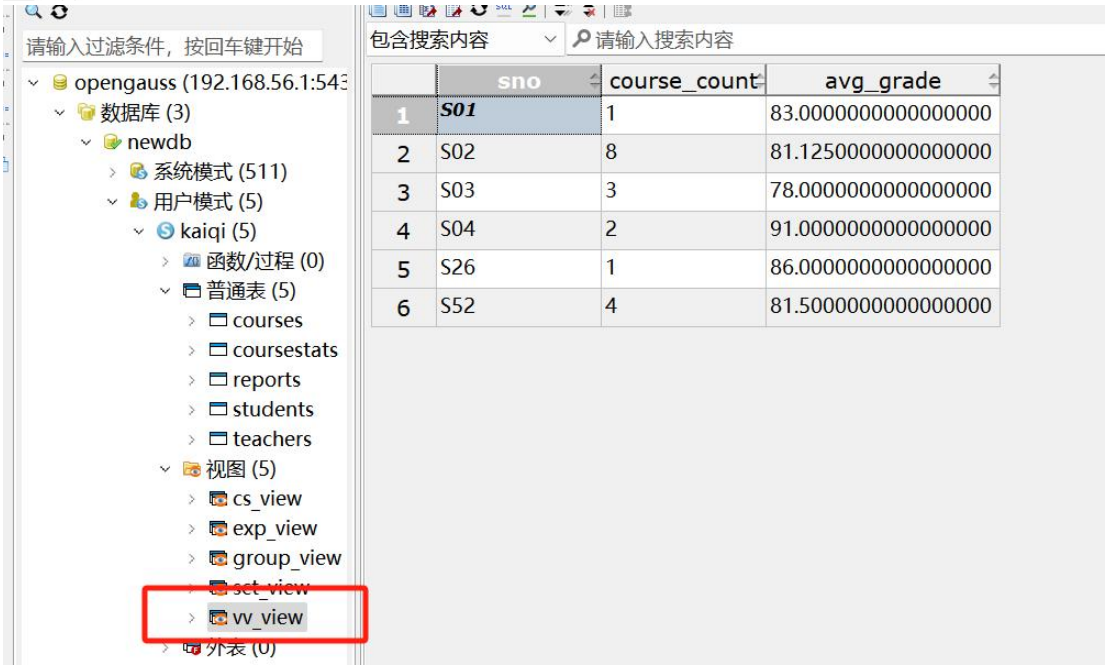


	sno	avg_grade
1	S01	83.0000000000000000
2	S02	77.1111111111111111
3	S03	78.0000000000000000
4	S04	91.0000000000000000
5	S26	65.5000000000000000
6	S52	81.5000000000000000

5) 创建一个基于视图的视图，基于（1）中建立的视图，定义一个包括学生编号、学生所选课程数目和平均成绩的视图 **VV_View**，并查询该视图。

基于之前创建的视图 CS_View，定义一个新视图 VV_View，该视图显示学生编号、学生选课数量以及其平均成绩。这展示了视图可以相互嵌套，以形成更复杂的查询结构。

```
1 CREATE VIEW VV_View AS
2 SELECT Sno, COUNT(Cno) AS Course_Count, AVG(score) AS Avg_Grade
3 FROM CS_View
4 GROUP BY Sno;
5
6 -- 查询视图
7 SELECT * FROM VV_View;
8
```



	sno	course_count	avg_grade
1	***S01***	1	83.0000000000000000
2	S02	8	81.1250000000000000
3	S03	3	78.0000000000000000
4	S04	2	91.0000000000000000
5	S26	1	86.0000000000000000
6	S52	4	81.5000000000000000

6) 创建一个物化视图 **MV_CS_View**，给出选课成绩合格的学生的编号、教师编号、所选课程号和该课程成绩，并查询该物化视图。向物化视图 **MV_CS_View** 中基表 **Reports** 插入数据**('S52','T05','C07',53)**，并查询视图 **MV_CS_View**。向物化视图 **MV_CS_View** 中基表 **Reports** 插入数据**('S52','T05','C03',83)**，并查询视图 **MV_CS_View**。

物化视图是将查询结果存储在物理表中，因此可以提高查询效率。在此操作中，我创建了一个与 CS_View 类似的物化视图 MV_CS_View，查询后发现物化表没有任何变化，查询资料后得知物化表需要手动更新

```sql
CREATE MATERIALIZED VIEW MV_CS_View AS
SELECT Sno, Tno, Cno, Score
FROM Reports
WHERE Score >= 60;

-- 查询物化视图
SELECT * FROM MV_CS_View;


-- 插入数据并查询
INSERT INTO Reports (Sno, Tno, Cno, Grade) VALUES ('S52', 'T05', 'C07', 53);
SELECT * FROM MV_CS_View;

INSERT INTO Reports (Sno, Tno, Cno, Grade) VALUES ('S52', 'T05', 'C03', 83);
SELECT * FROM MV_CS_View;
```

两次查询结果分别如下，没有任何变化

包含搜索内容 请输入搜索内容

	sno	tno	cno	score
1	S01	T01	C01	83.0
2	S02	T01	C01	75.0
3	S02	T05	C05	70.0
4	S02	T04	C06	83.0
5	S02	T05	C07	90.0
6	S02	T01	C08	83.0
7	S02	T02	C09	77.0
8	S02	T07	C10	83.0
9	S02	T06	C11	88.0
10	S03	T01	C08	63.0
11	S03	T02	C02	93.0
12	S03	T01	C01	78.0
13	S04	T06	C06	89.0
14	S04	T05	C05	93.0
15	S26	T04	C04	86.0
16	S52	T07	C10	91.0
17	S52	T06	C11	90.0
18	S52	T01	C08	64.0
19	S52	T02	C09	81.0

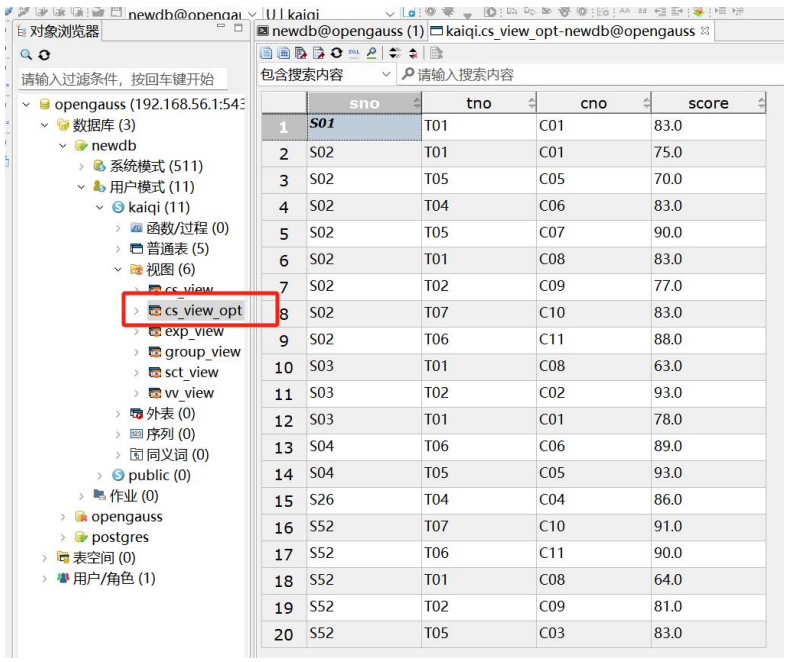
	sno	tno	cno	score
1	S01	T01	C01	83.0
2	S02	T01	C01	75.0
3	S02	T05	C05	70.0
4	S02	T04	C06	83.0
5	S02	T05	C07	90.0
6	S02	T01	C08	83.0
7	S02	T02	C09	77.0
8	S02	T07	C10	83.0
9	S02	T06	C11	88.0
10	S03	T01	C08	63.0
11	S03	T02	C02	93.0
12	S03	T01	C01	78.0
13	S04	T06	C06	89.0
14	S04	T05	C05	93.0
15	S26	T04	C04	86.0
16	S52	T07	C10	91.0
17	S52	T06	C11	90.0
18	S52	T01	C08	64.0
19	S52	T02	C09	81.0

7) 创建一个带 **WITH CHECK OPTION** 子句的视图 **CS_View_opt**，给出选课成绩合格的学生的编号、教师编号、所选课程号和该课程成绩，并查询该视图。**openGauss** 与 **SQLServer** 分别是如何处理的？**(**注：在实现 **SQL** 语句时，对于标准 **SQL** 之外的规定，每个 **DBMS** 处理方式不一样。**)**

WITH CHECK OPTION 确保了视图中的插入或更新操作必须满足视图的 WHERE 条件。在此视图中，所有插入或更新的记录必须满足成绩大于等于 **60** 的条件。否则，操作会被拒绝。

- **openGauss**：**WITH CHECK OPTION** 保证新插入或修改的记录符合 **WHERE** 条件，否则报错。
- **SQL Server**：同样通过 **WITH CHECK OPTION** 来确保插入或更新的数据满足视图的条件。

```
1  CREATE VIEW CS_View_opt AS
2  SELECT Sno, Tno, Cno, Score
3  FROM Reports
4  WHERE Score >= 60
5  WITH CHECK OPTION;
6
7  -- 查询视图
8  SELECT * FROM CS_View_opt;
9
10
```



	sno	tno	cno	score
1	S01	T01	C01	83.0
2	S02	T01	C01	75.0
3	S02	T05	C05	70.0
4	S02	T04	C06	83.0
5	S02	T05	C07	90.0
6	S02	T01	C08	83.0
7	S02	T02	C09	77.0
8	S02	T07	C10	83.0
9	S02	T06	C11	88.0
10	S03	T01	C08	63.0
11	S03	T02	C02	93.0
12	S03	T01	C01	78.0
13	S04	T06	C06	89.0
14	S04	T05	C05	93.0
15	S26	T04	C04	86.0
16	S52	T07	C10	91.0
17	S52	T06	C11	90.0
18	S52	T01	C08	64.0
19	S52	T02	C09	81.0
20	S52	T05	C03	83.0

8) 在 SQL Server 中建造好与 openGauss 一样的数据库及相关的基本表，并建立一个和(7)一样的视图，插入元组（'S52','T05','C07',51）到视图 CS_View_opt 中。分析对插入操作有什么影响。通过视图 CS_View_opt，把所有课程编号为 C01 的课程的成绩都减去 5 分。这个操作数据库是否会正确执行，为什么？如果加上 5 分（原来 95 分以上的不变）呢？在视图 CS_View_opt 删除编号 S03 学生的记录，会产生什么结果？【注：该步骤可在 SQL Server 中操作】

在 SQL Server 中创建与 openGauss 相同的视图，并测试插入操作。插入不符合条件（成绩小于 60）的数据将会失败，这表明 WITH CHECK OPTION 在不同数据库管理系统中都起到了数据验证的作用。

- 插入元组 'S52','T05','C07',51 到 CS_View_opt 中时，违反 WITH CHECK OPTION 条件，插入操作将失败。

```
UPDATE CS_View_opt
SET Score = Score - 5
WHERE Cno = 'C01';

```

此操作不会违反约束，因为减分操作后成绩可能仍然大于等于 60。

```
UPDATE CS_View_opt
SET Score = CASE
    WHEN Score < 95 THEN Score + 5
    ELSE Score
    END
WHERE Cno = 'C01';
```

对于 95 分以上的保持不变，95 以下的进行加分，此操作会成功执行，因为不会违反 WITH CHECK OPTION。

```
DELETE FROM CS_View_opt
WHERE Sno = 'S03';

```

删除操作只要符合条件可以执行，但可能受其他外键约束限制。

9) 删除视图 **SCT_View**、视图 **CS_View** 和视图 **CS_View_opt**，删除物化视图 **MV_CS_View**。思考物化视图与普通视图的区别。

删除创建的视图和物化视图，区分了普通视图和物化视图的不同：

- 普通视图：动态生成，查询时实时计算，不占用存储空间。
- 物化视图：将查询结果存储下来，能够提高查询效率，但占用存储空间，且需要定期刷新。

```
1 DROP VIEW SCT_View;
2 DROP VIEW CS_View;
3 DROP VIEW CS_View_opt;
4 DROP MATERIALIZED VIEW MV_CS_View;
5
```

三、　实验中遇到的问题及分析讨论

问：在向基表插入数据后，观察到物化表没有任何变化：

答：物化表数据固化，需要手动更新才能显示新的数据，用 **REFRESH MATERIALIZED VIEW MV_CS_View;**类似这样的指令即可。

```
2
3  -- 查询物化视图
4  SELECT * FROM MV_CS_View;
5
6  -- 插入数据并查询
7  INSERT INTO Reports (Sno, Tno, Cno, Grade) VALUES ('S52', 'T05', '
8  SELECT * FROM MV_CS_View;
9
10 INSERT INTO Reports (Sno, Tno, Cno, Grade) VALUES ('S52', 'T05', '
11 SELECT * FROM MV_CS_View;
```

运行时间：242 ms

消息　结果

包含搜索内容　请输入搜索内容

	sno	tno	cno	score
1	S01	T01	C01	83.0
2	S02	T01	C01	75.0
3	S02	T05	C05	70.0
4	S02	T04	C06	83.0
5	S02	T05	C07	90.0
6	S02	T01	C08	83.0
7	S02	T02	C09	77.0
8	S02	T07	C10	83.0
9	S02	T06	C11	88.0
10	S03	T01	C08	63.0
11	S03	T02	C02	93.0
12	S03	T01	C01	78.0
13	S04	T06	C06	89.0
14	S04	T05	C05	93.0
15	S26	T04	C04	86.0
16	S52	T07	C10	91.0
17	S52	T06	C11	90.0
18	S52	T01	C08	64.0
19	S52	T02	C09	81.0

查询提交时间：2024-09-16 1获取19行，共8 ms 可以编辑查询结果。

四、 实验结论

通过本次实验，我熟悉了视图的创建、查询、更新及删除操作，了解了视图与物化视图的区别，并学会了使用 **WITH CHECK OPTION** 子句确保数据完整性。实验中，通过使用视图简化了多表查询和数据汇总分析操作，同时验证了视图在不同数据库管理系统中的一致性行为。最终，我掌握了视图在数据更新和优化查询中的实际应用。

五、 教师批阅意见：

成绩评定：

教师签字：

年 月 日

六、 备注：